Assignment 5 - Advanced Programming 2026

Anton Bang Lejbølle & Jonas Dennis Hansen

October 2026

1 Introduction

For this assignment, we have identified some ambiguities in the problem text, particularly in question 3, where it is unclear whether the language is only defined for positive integers or whether negative integers should also be supported. This affects how expressions are generated and whether they can be printed and parsed back correctly.

Another consideration is the generation of variable names. We interpreted the requirements as allowing variable names of different lengths, provided that the required coverage criteria are satisfied. Instead of restricting all variable names to between two and four characters, our solution has a wider degree of randomness in variable length, including shorter and longer variable names, and therefore gives us a better chance of discovering potential edge cases.

2 Answers to questions

1. Can programs produced by your genExp loop infinitely when evaluated? If so, is it possible to avoid this?

This can happen when the generator creates Lambda expressions that apply a function to itself, causing evaluation to repeat indefinitely. To guarantee termination, we would need to restrict how Apply expressions are generated in genExp. For example we could limit function applications to simple Lambda expressions whose bodies and arguments are known to terminate, and prevent functions from repeatedly applying their arguments. These restrictions would need to apply recursively to all subexpressions. However, this would limit the variety of expressions generated excluding some valid expressions in the language that terminate normally.

2. Which of the four coverage requirements in expCoverage was hardest to satisfy, and what did you do to satisfy it?

For us the hardest of the four coverage requirement in expCoverage to satisfy was " Between 5% and 30% of expressions should have variable errors reported

by checkExp".

We satisfied it by adding an additional parameter of type [VName] to genExp, and by changing the Let and Lambda generator such that the variable name is added to this new parameter, to be used later in the second expression. Then we added cases for genExp such that when this new parameter is not empty then it sometimes generates Var expressions using a variable name from this list. We also changed the frequencies such that Var is only rarely generated with a variable that is not in scope.

3. Which counterexamples did parsePrinted produce? For each counterexample, which component (implementation, generator or property) did you fix, and why was that the right place to fix it?

These are the counterexamples that parsePrinted produced:

```
CstInt (-1)
```

This is a problem with the implementation of the parser, as it cannot parse negative integers even though our language is defined on all integers.

```
Var ""
```

This is a problem with the generator as "arbitrary" generates the empty string, but this is not a valid variable name in our language.

```
Var "e+\EOTb"
```

Again this is a problem with the generator as in our language variable names are defined as one alpha character optionally followed by alphanumeric characters.

```
Let "if" (CstBool False) (Var "if")
```

This is the final problem with the generator as variable names must not be in the list of keywords.

```
Div (TryCatch (CstInt (-6)) (CstInt 5)) (CstBool False)
```

This is a problem with the implementation of print as TryCatch was printed without parenthesis, so it got printed as "(try -6 catch 5 / false)", and since TryCatch binds tighter than Div, this would be parsed as "TryCatch (CstInt (-6) (Div (CstInt 5) (CstBool False))". Instead it should have gotten printed as "((try -6 catch 5) / false)".

```
Apply (CstBool False) (Apply (CstInt (-14)) (CstInt (-4)))
```

This is also a problem with the implementation of print, as Apply was also printed without parentheses, so this got printed as "false -14 -4", and since Apply is left associative, it gets parsed as "Apply (Apply (CstBool False) (CstInt (-14)) (CstInt (-4)))". Instead it should have gotten printed as "false (-14 -4)".

4. Which assumption underlying checkExp is not justified, and which counterexample did onlyCheckedErrors produce to demonstrate this?

The assumption underlying checkExp that is not justified, is the assumption that all errors in the first expression of TryCatch should be ignored.

This assumption is not justified because when the first expression of the TryCatch is a Lambda expression that contains errors in the body expression, then the Lambda expression in itself will evaluate without a problem, but as soon as the Lambda expression is applied then it will evaluate to the error that was in the body expression. Since checkExp ignores all errors found in the first expression of TryCatch, then it will ignore the error in the body of the Lambda expression, even though the expression will evaluate to this error.

The counterexample that onlyCheckedErrors produced to demonstrate this was:

```
Apply (TryCatch (Lambda "Tz" (Var "xu")) (CstBool False)) (CstBool True)
```

5. Your generator satisfies expCoverage. Describe a class of expressions that your generator will nevertheless produce essentially never, but that would be worth testing. How would you change either the generator or expCoverage to cover it?

One class of expressions that our generator will very rarely produce, but would be worth testing is expressions where a variable name is used outside of the scope it is defined by a Let or Lambda expression. Similarly our generator will also rarely produce expressions where the same variable name is defined multiple times in the same scope. Since our variable names are produced randomly and since we rarely produce Var expressions with a variable name that is not in scope, it will very rarely be the case that the same variable name will be randomly generated twice in the same expressions.

These classes could be worth testing to make sure that our variables are not defined globally, can only be used in the local scope, and that defining the same variable again correctly overwrites the old definition.

To cover it we could change the generator to initially generate a list of variable names using our variable name generator, then in our when generating variable names it should either use a name in this list or generate a random new variable name.